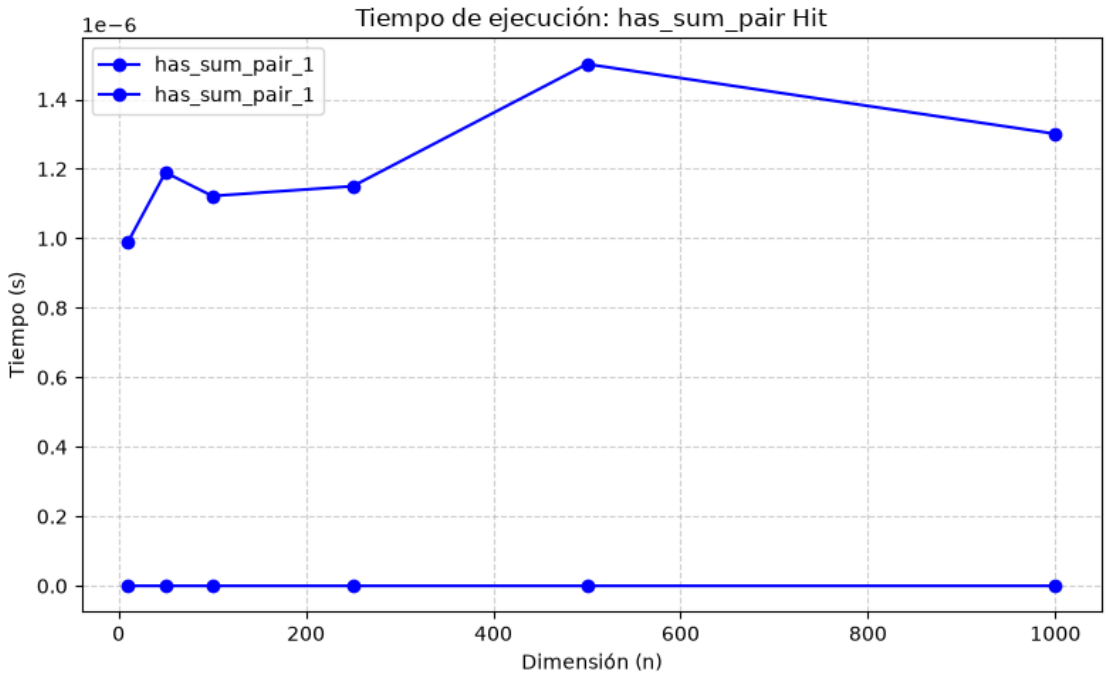
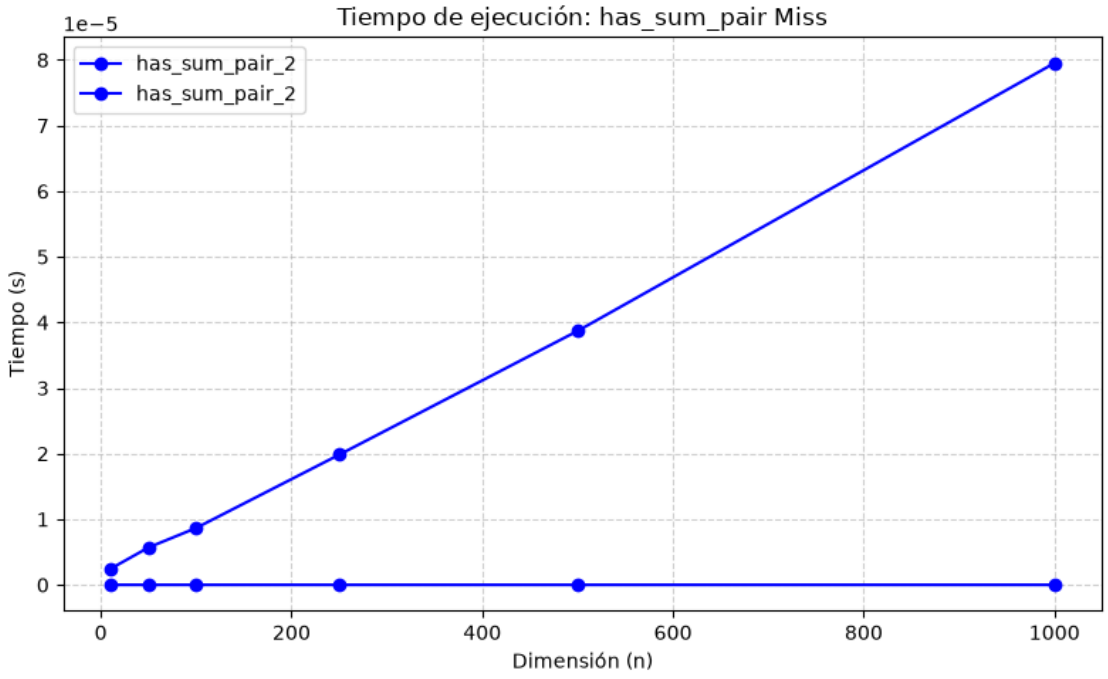
Fichero de respuestas y gráficas de Práctica de
Algoritmia 1
11-10-2026

Gráficas Apartado I.A





Cuestiones Apartado I.A:

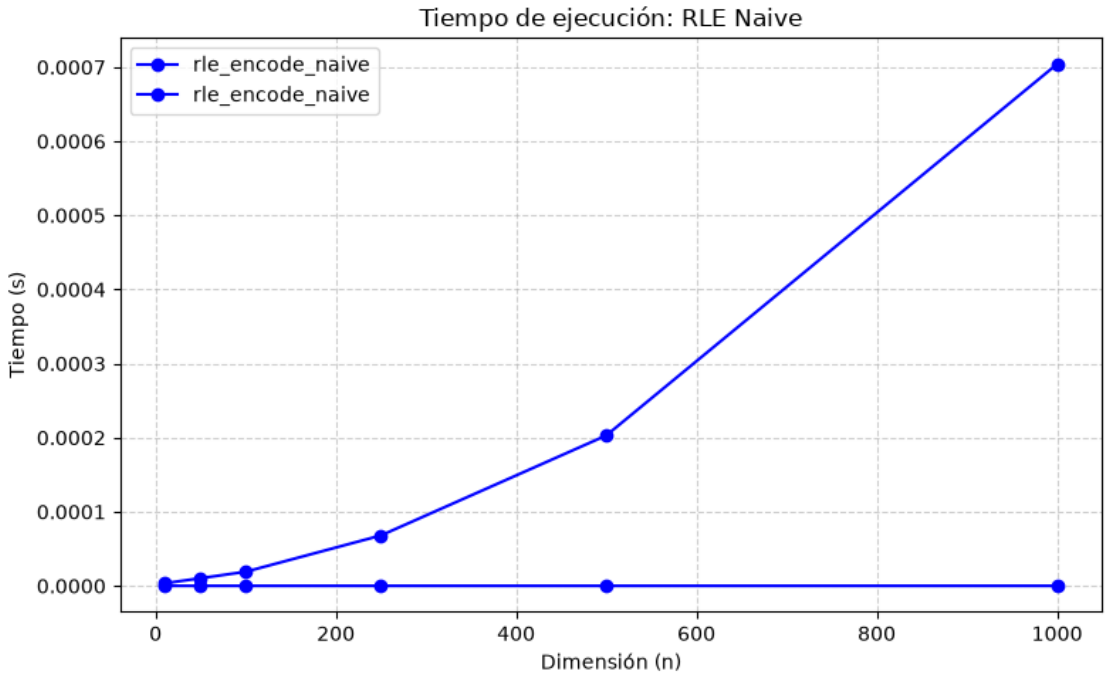
1. ¿Cómo varía el tiempo de ejecución en ambos casos a medida que crece n? ¿Mantiene la función una complejidad temporal de $O\,n$ en ambos escenarios?

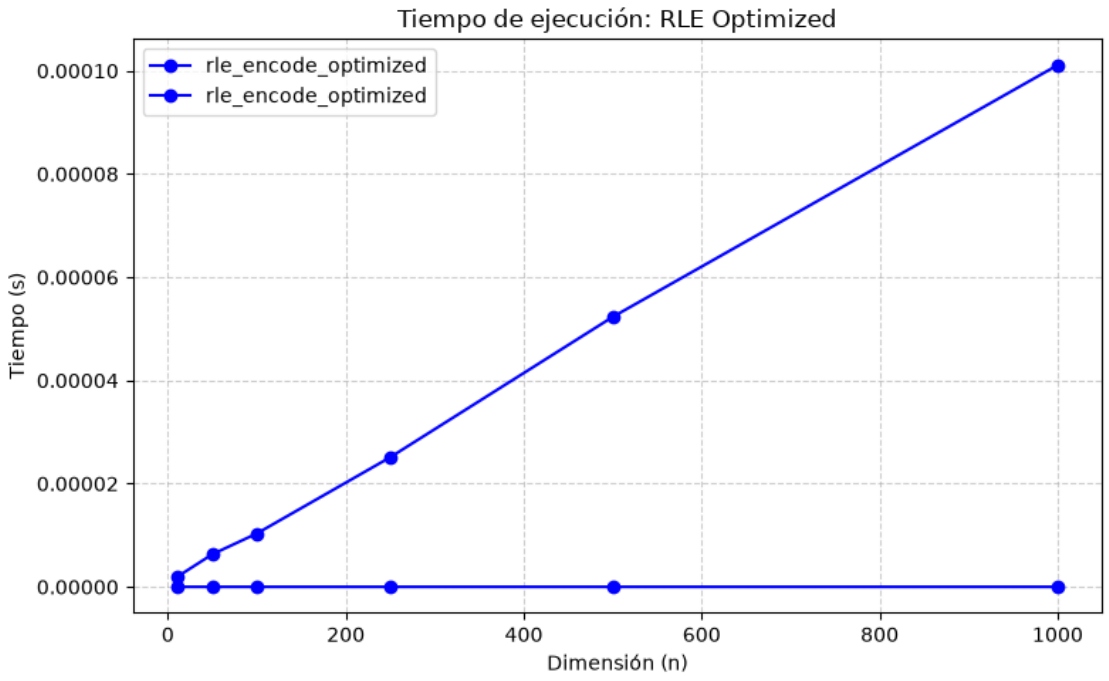
El tiempo de ejecución se mantiene prácticamente constante en el mejor caso (Hit), lo que indica una complejidad temporal de $O(1)$. Esto ocurre porque el

algoritmo se detiene al encontrar la suma buscada. En el caso contrario, el tiempo de ejecución crece de forma lineal y proporcional al tamaño de n, indicando una complejidad de $O(n)$. Al no existir el par que suma el objetivo, la función tiene que recorrer toda la estructura de datos para confirmarlo.

Por tanto, la complejidad varía entre $O(1)$ (Hit) y $O(n)$ (Miss) dependiendo de los datos de entrada.

Gráficas Apartado I.B





Fichero de respuestas y gráficas de Práctica de
Algoritmia 1
11-10-2026

Cuestiones Apartado I.B:

1. ¿Cuál es el comportamiento temporal de ambas funciones al crecer n? ¿Qué impacto tienen las operaciones internas de creación/modificación de listas?

La función ineficiente (naive) muestra una curva de crecimiento parabólica, (complejidad $O(n^2)$). Por el contrario, la gráfica optimizada (optimized) es una línea recta casi perfecta (complejidad lineal $O(n)$).

En la versión ineficiente (naive), se utilizan operaciones de "operador + concatenador de listas", que obligan a crear una nueva lista y copiar todos los elementos anteriores en memoria en cada iteración. La versión optimizada utiliza operaciones "append in-place", las cuales simplemente añade elementos al final del bloque de memoria.

Cuestiones Apartado II.C:

1. Sin darnos cuenta, en nuestro algoritmo de encontrar CCs podemos pasar listas con ramas repetidas o donde los vértices coinciden con los de una que ya está, aunque en orden inverso. ¿Afectará esto al resultado del algoritmo? ¿Por qué?

No afecta el resultado final del algoritmo porque la función usada find devolvería el mismo resultado y debido a la condicional no se produciría.

2. Argumentar que nuestro algoritmo de encontrar componentes conexas es correcto, esto es, que a su final en los distintos subconjuntos disjuntos se encuentran los vértices de las distintas componentes del grafo dado.

Al comienzo del algoritmo el algoritmo comprueba a cara arista (u,v) que u y v pertenecen a subconjuntos distintos y si son disjuntos fusiona sus árboles correspondientes y se podría concluir que es correcto si dos vértices pertenecen al mismo árbol

3. El tamaño de un grafo no dirigido viene determinado por el número n de nodos y la longitud de la lista l de ramas. Estimar razonadamente en función de ambos el coste del algoritmo de encontrar las componentes conexas mediante conjuntos disjuntos.

La inicialización tiene un coste de O(n), mientras que el procesamiento de las aristas depende de la longitud de la lista de ramas |l| y como lg(n) del coste de find por lo que su coste sería de O(n + |l|*lg(n))